# Московский авиационный институт (национальный исследовательский университет)

## Факультет информационных технологий и прикладной математики

## Кафедра вычислительной математики и программирования


**Лабораторная работа №3 по курсу «Информационный поиск»**


Студент: А. М. Голев
Преподаватель: А. А. Кухтичев
Группа: М8О-401Б

Дата:
Оценка:
Подпись:

**Москва, 2026**

# Лабораторная работа №3 «Токенизация»

Необходимо реализовать процесс разбиения текстов документов на токены, который потом будет использоваться при индексации. Для этого потребуется выработать правила, по которым текст делится на токены. Необходимо описать их в отчёте, указать достоинства и недостатки выбранного метода. Привести примеры токенов, которые были выделены неудачно, объяснить, как можно было бы поправить правила, чтобы исправить найденные проблемы.

В результатах выполнения работы нужно указать следующие статистические данные:

- количество токенов;
- среднюю длину токена.

Кроме того, нужно привести время выполнения программы, указать зависимость времени от объёма входных данных. Указать скорость токенизации в расчёте на килобайт входного текста. Является ли эта скорость оптимальной? Как её можно ускорить?

# 1 Описание

## 1 Общее устройство

Токенизатор — программа `lexer.cpp` на C++17 (компилятор Apple Clang 21, флаги `-std=c++17 -O2 -Wall`), вся логика — класс `MusicTokenizer`. Сборка и запуск — через `Makefile`: `make run` прогоняет токенизатор по обоим корпусам из первой лабораторной работы (`lyrics_corpus` и `musicbrainz_corpus`, HTML-документы), `make test` — синтетический тест производительности. Программа рекурсивно обходит каталоги, отбирает файлы `.html`, `.htm`, `.txt` и считает статистику: количество токенов, символов, среднюю длину токена, время и скорость токенизации.

## 2 Препроцессинг: извлечение текста из HTML

Перед разбиением на токены из документа извлекается текстовый контент (функция `extractTextFromHTML`):

- вырезаются содержимое тегов `<script>`, `<style>` и HTML-комментарии `<!- ->` целиком;
- сами теги удаляются; после закрывающего тега вставляется пробел, чтобы слова из соседних элементов не склеивались (`<b>bold</b>text` → `bold text`);
- HTML-сущности декодируются: `<` → <, `>` → >, `&` → &, `"` → ", `'` → ’;
- переводы строк заменяются на пробелы.

## 3 Правила токенизации

Разбиение на токены — по разделителям, с последующей нормализацией и фильтрацией каждого токена.

**Разделители.** ASCII-пробельные символы (пробел, табуляция, переводы строк) и все знаки ASCII-пунктуации от `!` до `~` (строка `delimiters` в коде). Кроме них — юникодные разделители, определяемые по ведущим байтам UTF-8 последовательности (функция `separatorLength`):

- `U+0080-U+00BF` (байты `C2 xx`): C1-управляющие (мусор от неправильно декодированных CP1252-кавычек), неразрывный пробел `U+00A0`, испанские `U+00A1`/`U+00BF`, кавычки `«»`, знак ударения `U+00B4`, пилкроу `U+00B6`;

- U+2000-U+207F (байты E2 80/81 xx): юникодные пробелы (в том числе U+2005, U+205F), тире -- -, типографские кавычки ‘’ “”, многоточие, двойную вертикаль U+2016;
- U+2600-U+27BF (байты E2 98-9E xx): музыкальные символы U+266A-U+266C и прочие значки;
- U+3000-U+303F (байты E3 80 xx): CJK-пунктуация;
- эмодзи U+1F000+ (байты F0 9F xx xx): в корпусе MusicBrainz каждый документ начинается с U+1F3A4 и содержит U+1F4CA.

Буквы — ASCII, кириллица (D0/D1 xx), акцентированная латиница (C3 xx), корейский, японский — остаются внутри токенов.

**Нижний регистр.** ASCII — тривиально; кириллица — вручную по байтам UTF-8: D0 90-AF (А–Я) → +0x20, Ё (U+0401) → ё (U+0451); блок D1 80-9F (р–я и расширенная кириллица) уже строчные.

**Защита аббревиатур.** Перед разбиением вхождения музыкальных аббревиатур (feat., ft., vs., remix, ep, lp, bpm, ...), обрамлённые пробелами, помечаются суффиксом _PROTECTED, который вырезается из готового токена.

**Очистка токена** (cleanToken): обрезка краевых пробелов; декодирование сущностей; отбрасывание токенов из одних цифр длиннее 4 символов (годы и счётчики оставляем, ID и таймстампы — нет); отбрасывание односимвольных токенов, кроме местоимений i, a, я.

**Стоп-слова и мусор** (shouldSkipToken, включается для корпуса lyrics): английские и русские стоп-слова (the, and, это, и, ...); URL (http://, www.), email и домены (.com, .ru).

## 4 Достоинства и недостатки метода

**Достоинства:** правила просты и детерминированы; разбиение по разделителям не требует словарей и работает для любого языка; нормализация регистра и фильтры (цифры, URL, стоп-слова) сразу снижают шум индекса; обработка юникода на уровне байтов UTF-8 не требует внешних библиотек.

**Недостатки:** список разделителей неполон и расширялся по факту (обнаружено на корпусе); апостроф и дефис — разделители, поэтому couldn’t и b-side режутся; фильтры эвристичны (число до 4 цифр — «хорошее», ID из букв и цифр — нет); защита аббревиатур фактически не работает: точки и дефисы внутри feat. и b-side всё равно режут токен; средняя длина считается по всему тексту, включая пробелы между токенами.

## 5 Примеры неудачных токенов

Примеры из реального прогона по корпусам и способы исправления правил:

| Токен(ы) | Проблема | Как поправить правила |
|---|---|---|
| `000ee19e`, `f1db`, `4e22`, `b8a4` | MBID (UUID) из MusicBrainz разрезан дефисами на 5 кусков | распознавать шаблон UUID (8-4-4-4-12 hex) и не токенизировать его или отбрасывать целиком |
| `couldn` (из `couldn't`) | апостроф — разделитель, `t` отброшен как односимвольный | не считать апостроф разделителем внутри слова (между буквами): `couldn't` — один токен |
| `side` (из `b-side`) | `b` отброшен (1 символ) | аналогично: дефис между буквами — часть слова |
| `the antidote this time` одним токеном (до исправления) | в текстах песен встречается юникодный пробел `U+205F`, который не был разделителем — слова склеивались в один токен | добавить юникодные пробелы `U+2000-U+205F` в разделители (сделано) |
| `U+1F3A4`, `U+1F4CA` (30000 шт.) | эмодзи из шаблона страниц MusicBrainz попадали в токены | считать эмодзи разделителями (сделано) |
| `2026`, `10`, `03`, `16`, `31` | дата обкачки из подвала страницы режется на шумовые токены | вырезать служебные блоки шаблона (footer) при извлечении текста |

## 6 Статистика запуска

Статистика приведена для корпуса из 30000 документов (по 15000 из каждого источника: Lyrics.ovh и MusicBrainz; 260 МБ сырого HTML):

| Показатель | Значение |
|---|---|
| Обработано файлов | 30000 |
| Извлечено текста из HTML | 41,9 МБ |
| Количество токенов | 5 503 500 |
| Токенов на документ | 183 |
| Средняя длина токена | 7,61 символа |
| Время токенизации | 3,4 с |
| Общее время (с чтением файлов) | 7,6 с |
| Скорость токенизации | 11,7 МБ/с |

## 7 Время и скорость

Зависимость времени от объёма входных данных (синтетический тест `-test`, текст наращивается повторением одного фрагмента):

| Размер текста | Время | Скорость | Токенов/с |
|---|---|---|---|
| 34 КБ | 0,0016 с | 20,5 МБ/с | 2,19 млн |
| 172 КБ | 0,0074 с | 22,7 МБ/с | 2,44 млн |
| 688 КБ | 0,037 с | 18,3 МБ/с | 1,96 млн |
| 3438 КБ | 0,454 с | 7,4 МБ/с | 0,79 млн |

До ∼700 КБ время растёт практически линейно (скорость стабильна 18–23 МБ/с). На мегабайтных текстах скорость падает: перед разбиением текст 17 раз сканируется find/replace-проходами защиты аббревиатур, каждый `replace` переносит хвост строки — вклад растёт с длиной.

Скорость нельзя считать оптимальной. Основные резервы ускорения:

- заменить 17 проходов `find/replace` по всему тексту одним проходом с проверкой аббревиатур на лету (или убрать механизм — он инертен);
- не копировать строки: сейчас текст копируется в `cleanText`, `processedText`, `currentToken` посимвольно; токены можно нарезать указателями (string_view) на исходный буфер;
- классификацию байтов (разделитель/буква) — таблицей из 256 значений вместо `delimiters.find(c)`;
- многопоточность: файлы независимы, обработка легко делится на потоки (OpenMP) практически без синхронизации.

По порядку величины это дало бы десятки МБ/с на одном ядре и сотни МБ/с на многопоточной версии — с запасом быстрее текущих 11,7 МБ/с.

# 2 Исходный код

Программа `lexer.cpp` состоит из класса `MusicTokenizer`. Основные блоки: извлечение текста из HTML, определение длины юникодного разделителя, основной цикл токенизации, приведение кириллицы к нижнему регистру, очистка и фильтрация токенов, сбор статистики.

Юникодные разделители — по ведущим байтам UTF-8 последовательности (многобайтовые символы нельзя сравнивать с `char`-литералами):

```
size_t separatorLength(const std::string& text, size_t pos) const {
    unsigned char c = static_cast<unsigned char>(text[pos]);
    if (c == 0xC2 && pos + 1 < text.length())
        return 2; // U+0080-U+00BF: C1 junk, nbsp, punct
    if (c == 0xE2 && pos + 2 < text.length()) {
        unsigned char c2 = static_cast<unsigned char>(text[pos + 1]);
        // U+2000-U+207F: spaces, dashes, quotes; U+2600-U+27BF: symbols
        if (c2 == 0x80 || c2 == 0x81 || (c2 >= 0x98 && c2 <= 0x9E))
            return 3;
        return 0;
    }
    if (c == 0xF0 && pos + 3 < text.length()) {
        // U+1F000+: emoji
        if (static_cast<unsigned char>(text[pos + 1]) == 0x9F)
            return 4;
    }
    return 0;
}
```

Извлечение текста из HTML: вырезание `<script>`/`<style>`/`комментариев` по закрывающим последовательностям, пробел после закрывающего тега:

```
if (c == '>') {
    if (inComment) {
        if (i >= 2 && html.substr(i - 2, 2) == "--")
            inComment = false, inTag = false;
        continue;
    }
    if (inScript) {
        if (i >= 8 && html.substr(i - 8, 8) == "</script")
            inScript = false, inTag = false;
        continue;
    }
    if (inStyle) {
        if (i >= 7 && html.substr(i - 7, 7) == "</style")
            inStyle = false, inTag = false;
        continue;
    }
    if (inTag) {
        inTag = false;
```

```
        if (closingTag) // </div> etc -> word boundary
            shouldAddSpace = true, closingTag = false;
        continue;
    }
}
```

Основной цикл токенизации: накопление символов до разделителя, пропуск байтов юникодного разделителя, нормализация и фильтрация:

```
for (size_t i = 0; i < processedText.length(); i++) {
    char c = processedText[i];
    if (c == '_' && processedText.compare(i, 10, "_PROTECTED") == 0) {
        i += 9; // skip abbreviation protection marker
        continue;
    }
    size_t sepLen = separatorLength(processedText, i);
    if (isDelimiter(c) || sepLen > 0) {
        if (!currentToken.empty()) {
            std::string processed = toLowerWithCyrillic(currentToken);
            processed = cleanToken(processed);
            if (!processed.empty() &&
                (!removeStopWords || !shouldSkipToken(processed)))
                tokens.push_back(processed);
            currentToken.clear();
        }
        if (sepLen > 0) i += sepLen - 1; // skip separator bytes
    } else {
        currentToken += c;
    }
}
```

Приведение кириллицы к нижнему регистру по байтам UTF-8 (`D0 90-AF` — А–Я, `D0 81` — Ё):

```
if (isCyrillic(c) && i + 1 < str.length()) {
    unsigned char c2 = static_cast<unsigned char>(str[i + 1]);
    if (c == 0xD0 && c2 == 0x81) { // U+0401 Yo -> U+0451 yo
        result.push_back(0xD1); result.push_back(0x91); i++;
    } else if (c == 0xD0 && c2 >= 0x90 && c2 <= 0xAF) {
        result.push_back(c); result.push_back(c2 + 0x20); i++;
    } else { // D1 xx: already lowercase
        result.push_back(c); result.push_back(c2); i++;
    }
} else if (c >= 'A' && c <= 'Z') {
    result.push_back(c + 32);
}
```

# 3 Выводы

Выполнив третью лабораторную работу по курсу «Информационный поиск», я реализовал токенизацию текстов документов — компонент, на котором будет строиться индекс следующих лабораторных работ. Основное, что я узнал нового:

- выбор правил токенизации — компромисс: простое разбиение по разделителям предсказуемо и не требует словарей, но апострофы, дефисы и составные идентификаторы (UUID) режутся, а список разделителей приходится расширять по факту на реальном корпусе;
- токенизация реального корпуса — это в первую очередь препроцессинг: из 260 МБ сырого HTML в токены попадает 41,9 МБ текста, и без корректного вырезания `<script>`/`<style>`/комментариев статистика оказывается мусорной;
- UTF-8 нельзя обрабатывать как `char`: многобайтовые символы не помещаются в символьные литералы, кириллицу нужно приводить к нижнему регистру по байтам (и легко ошибиться: `ё` и `б` отличаются одним байтом), а юникодные пробелы (U+2000–U+205F) склеивают слова, если не считать их разделителями;
- эвристические фильтры (длина токена, «только цифры», стоп-слова) дёшевы и убирают заметный шум, но каждый — источник ложных срабатываний: `b-side` теряет `b`, таймстампы режутся в даты;
- скорость токенизации (11,7 МБ/с) не оптимальна и упирается не в сам проход по тексту, а в лишние копии строк и 17 find/replace-проходов защиты аббревиатур; ускорение — таблица классов символов, нарезка токенов без копирования и многопоточность по файлам.

Итог: из 30000 документов корпуса выделено 5,5 млн токенов (183 на документ, средняя длина 7,61 символа) за 7,6 с. Полученный поток токенов готов к использованию при построении индекса в следующей лабораторной работе.

# Список литературы


[1] Маннинг, Рагхаван, Шютце *Введение в информационный поиск* — Издательский дом «Вильямс», 2011. Перевод с английского: доктор физ.-мат. наук Д. А. Клюшина — 528 с. (ISBN 978-5-8459-1623-4 (рус.))

[2] Список использованных источников оформлять нужно по ГОСТ Р 7.05-2008

[3] The Unicode Standard, Version 15.0 — Chapter 2. General Structure; Chapter 3. Conformance // https://www.unicode.org/versions/Unicode15.0.0/

[4] RFC 3629. UTF-8, a transformation format of ISO 10646 // https://www.rfc-editor.org/rfc/rfc3629

[5] cppreference.com — C++ reference // https://en.cppreference.com

[6] HTML Standard — 13.2 Parsing HTML documents // https://html.spec.whatwg.org/multipage/parsing.html

[7] Lyrics.ovh API // https://api.lyrics.ovh

[8] MusicBrainz API Documentation // https://musicbrainz.org/doc/MusicBrainz_API